



컨테이너 개발 환경 세팅

학습 목표

- 내 컴퓨터의 도커 엔진이 정상 작동하는지 직접 확인합니다
- 공인 레지스트리에서 최신 Ubuntu LTS 이미지를 내려받습니다
- 명령어를 바로 주고받는 대화형 컨테이너를 띄우고 내부 셸에 접속합니다
- 컨테이너를 끄지 않고 안전하게 백그라운드로 분리하고 다시 연결합니다

운영체제의 핵심인 리눅스 커널과 도커 엔진의 동작 원리를 이해하고, `docker` 명령어로 첫 컨테이너를 실행·접속·분리하는 기본 조작을 숙달하는 것이 본 차시의 목표입니다.

핵심 개념 및 구조

컨테이너란 무엇인가

컨테이너는 컴퓨터를 통째로 새로 만드는 대신, 이미 켜져 있는 **리눅스 핵심 두뇌(커널)**를 함께 쓰면서 필요한 프로그램만 독립된 방에서 따로 돌리는 기술입니다.

가상 머신처럼 부팅부터 다시 시작하는 무거운 방식이 아니라, 이미 돌아가는 컴퓨터 안에서 **벽을 세우고 문을 잠가 나만의 독립 공간을 만드는 방식**입니다.



두뇌(커널)는 함께 공유
부팅 시간 제로



방(사용자 공간)은 철저히 격리
독립된 파일과 네트워크



일반 프로그램처럼 즉시 실행
1초 만에 켜고 끄기

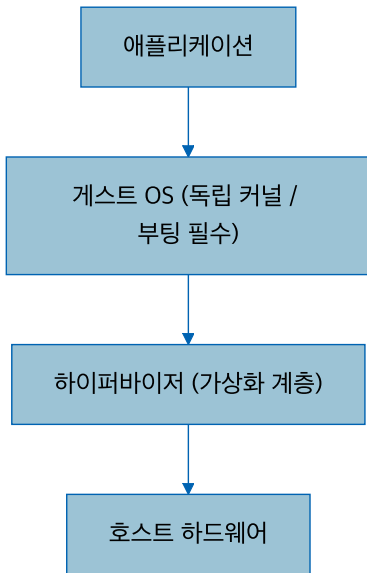
컨테이너 vs 가상 머신(VM)

비교 항목	가상 머신 (VM)	도커 컨테이너
현실 개념	땅부터 기둥까지 새로 짓는 단독주택	수도와 전기를 함께 쓰는 독립 원룸
격리 방식	하드웨어 수준 (OS 커널까지 통째 가상화)	운영체제 프로세스 수준 (호스트 커널 공유 격리)
시작 속도	수십 초 ~ 수 분 (부팅 과정 필수)	1초 안팎 (프로세스 즉시 시작)
컴퓨터 자원 소모	큼 (운영체제마다 독립 메모리 할당)	아주 적음 (프로그램이 쓰는 만큼만 사용)
배포 크기	수 GB ~ 수십 GB OS 이미지	수십 MB ~ 수백 MB 압축 이미지

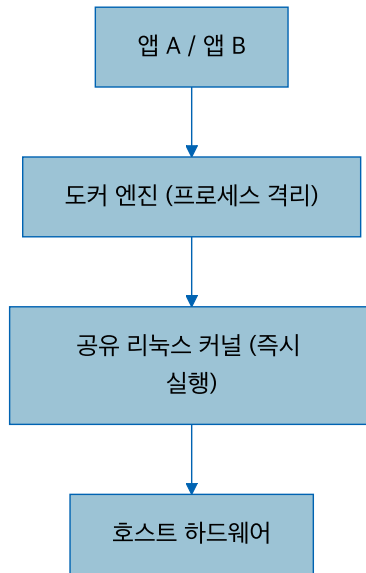
단독주택(VM)은 독립적이지만 짓고 관리하기 무겁습니다. 반면 원룸(컨테이너)은 건물 설비(커널)를 공유하므로 가볍고 순식간에 입주할 수 있습니다.

구조로 보는 VM과 컨테이너

가상 머신 (단독주택)



도커 컨테이너 (독립 원룸)



컨테이너는 무거운 게스트 운영체제를 두지 않고, 호스트 커널 위에서 **프로세스에 칸막이만 쳐서** 가볍게 실행합니다.

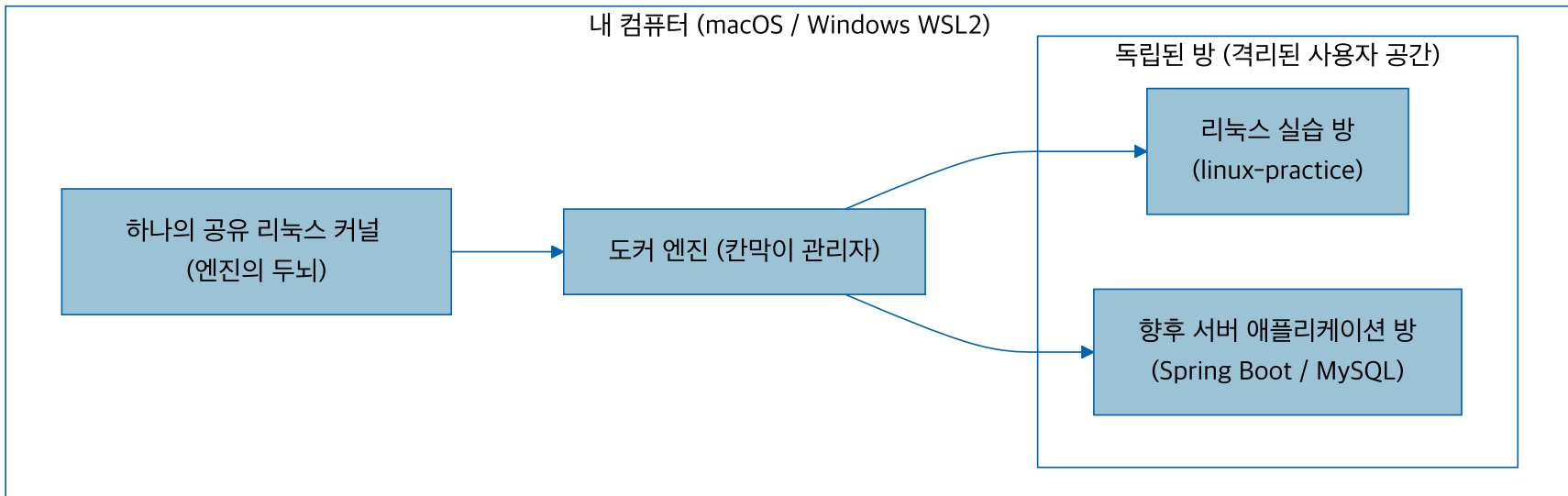
격리는 어떻게 만들어질까?

리눅스 커널이 원래 갖고 있던 두 가지 기술을 조합해 "완벽히 격리된 방"을 만듭니다.

커널 기능	담당 역할	현실 개념
Namespace	보이는 세상의 분리	내 방의 암막 커튼과 벽 — 내 방 안에서는 다른 방 사람(프로세스, 파일, 네트워크)이 보이지 않습니다.
cgroups	사용 자원의 제한	각 방의 전기와 수도 차단기 — 한 방에서 전기를 100% 써서 건물 전체가 마비되지 않도록 한도를 정합니다.

도커 컨테이너는 전에 없던 새로운 기술이 아니라, 리눅스 커널의 칸막이 기능(Namespace)과 계량기 기능(cgroups)을 누구나 쓰기 쉽게 포장해 준 프로그램입니다.

컨테이너 실행 구조



macOS · Windows에서는 어떻게 작동할까?

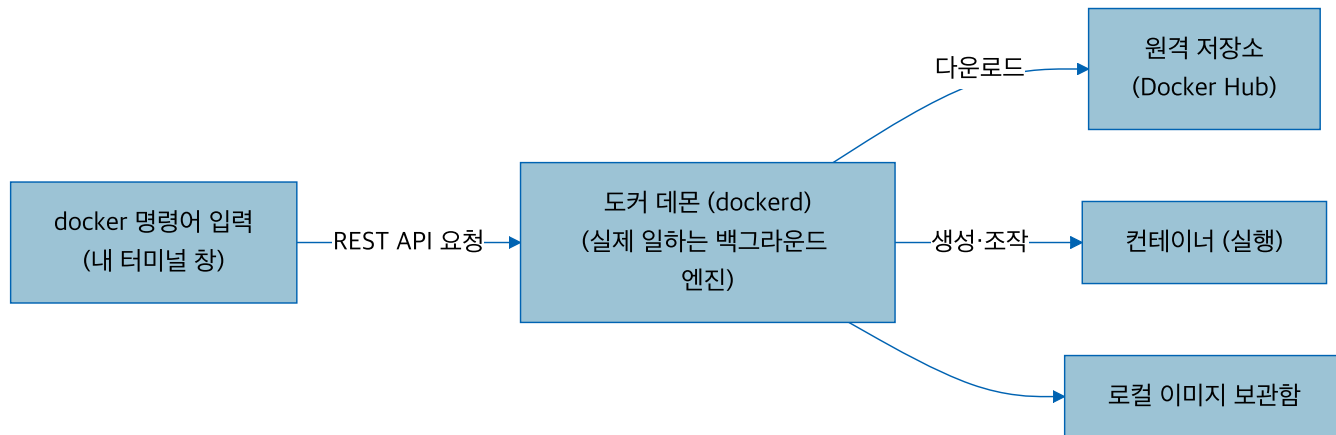
컨테이너는 리눅스 커널이 필수인데, 윈도우와 맥은 리눅스가 아닙니다. 그래서 **아주 얇은 경량 리눅스 가상머신을 바닥에 한 겹 깔고** 그 위에서 컨테이너를 띄웁니다.

운영체제	리눅스 커널을 준비하는 방식	우리가 주의할 점
Windows	WSL2 가 백그라운드에서 실제 마이크로소프트 공식 리눅스 커널 구동	BIOS 가상화나 WSL2가 꺼져 있으면 도커가 아예 켜지지 않습니다
macOS	Rancher Desktop이 내부 경량 가상머신으로 리눅스 커널 제공	처음 실행할 때 리눅스 환경을 켜느라 몇 초간 준비 시간이 걸립니다

Rancher Desktop 설정에서 런타임을 **Moby (dockerd)**로 선택하는 이유도, 우리가 배우는 표준 **docker** 명령어가 이 엔진과 대화하기 때문입니다.

우리가 입력한 명령어는 어디로 갈까?

터미널에서 `docker`를 쳐도 명령어가 직접 컨테이너를 만들지 않습니다. 우리가 쓰는 터미널은 **말을 전하는 심부름꾼 (Client)**이고, 실제 무거운 작업은 백그라운드에서 대기하는 **일꾼 데몬 (Server)**이 처리합니다.



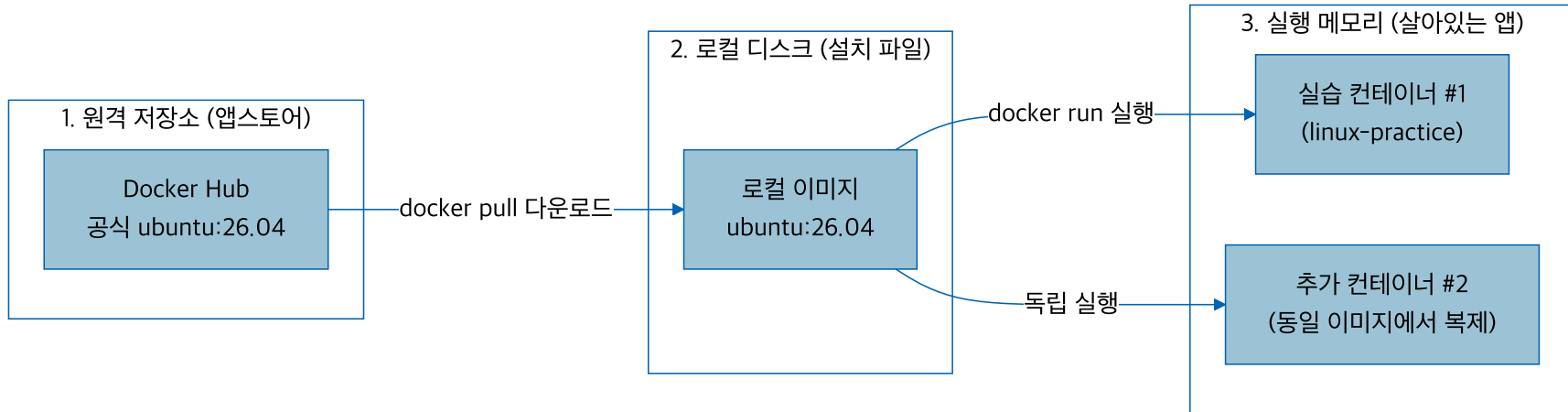
`docker version` 실행 시 Client와 Server 정보가 함께 출력되는 이유입니다. Server 정보가 표시되지 않는다면, 명령을 전달하는 CLI는 정상 동작하지만 실제 작업을 수행할 데몬이 구동되지 않은 상태입니다.

도커의 3대 핵심 구성 요소

요소	역할 및 특성	현실 개념
이미지 (Image)	프로그램 실행에 필요한 코드와 환경을 하나로 고정해 담은 읽기 전용 설계도	스마트폰 앱 설치 파일 (.apk), 밀키트 포장팩
컨테이너 (Container)	이미지를 바탕으로 메모리 위에 살아 움직이게 띄운 독립 실행체	화면에서 켜진 앱 화면, 조리되어 완성된 요리
레지스트리 (Registry)	전 세계의 검증된 이미지들을 모아두고 공유하는 온라인 저장소	스마트폰 앱스토어, 온라인 마켓

스마트폰에서 **앱스토어(레지스트리)**에서 **설치 파일(이미지)**을 내려받아 **앱(컨테이너)**을 켜는 것과 100% 똑같은 원리입니다.

3대 요소가 연결되는 전체 흐름



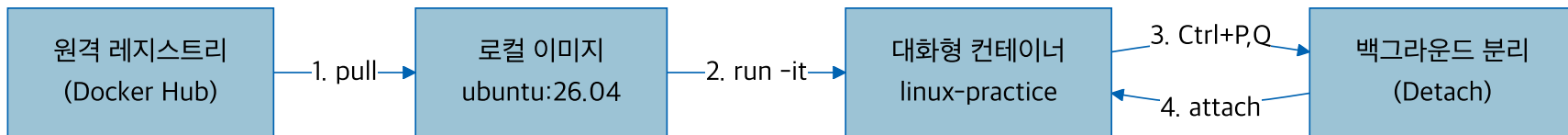
하나의 설계도(이미지)만 있으면 컴퓨터 자원이 허용하는 한 수십 개의 독립된 컨테이너를 몇 초 만에 복제해서 띄울 수 있습니다.

컨테이너의 상태 변화: 멈춤과 삭제의 차이

상태	어떤 상태인가?	내부 데이터	현실 개념
<code>Created</code>	생성되었으나 아직 실행 전	보존됨	전등이 꺼진 빈 방
<code>Up</code> (Running)	프로세스가 정상 실행 중	보존됨	사람이 활동 중인 방
<code>Exited</code>	작업이 끝나 프로세스가 종료됨	안전 유지	문만 잠가둔 방
삭제됨 (<code>rm</code>)	컨테이너가 완전히 제거됨	영구 삭제	방 자체를 철거함

대화형 셸에서 `exit`를 입력하면 컨테이너 프로세스가 종료됩니다(`Exited`). 실행 상태를 유지한 채 빠져나오려면 `Detach`(`Ctrl + P`, `Ctrl + Q`)를 사용합니다.

실습 전체 흐름: 이미지 다운로드부터 세션 분리까지



1. 이미지 다운로드

```
docker pull  
ubuntu:26.04
```

공식 레지스트리에서 계층 단위로 내려받는다

2. 로컬 보관 확인

```
docker images ubuntu
```

읽기 전용 설계도로 로컬에 남는다

3. 컨테이너 실행

```
docker run -it  
ubuntu:26.04 bash
```

이미지에서 독립 프로세스를 띄우고 셸에 접속한다

4. 세션 분리·재접속

```
Ctrl + P, Ctrl + Q /  
docker attach
```

종료하지 않은 채 빠져나오고 다시 붙는다

실습 단계

왜 Rancher Desktop인가

같은 `docker` 명령어를 쓰더라도 데스크톱 프로그램마다 라이선스와 런타임 선택 범위가 다릅니다.

도구	라이선스	런타임 선택	지원 플랫폼	특징 및 선택 이유
Docker Desktop	일정 규모 이상 기업은 유료 구독	<code>dockerd</code> 고정	Win / macOS / Linux	기업 도입 시 라이선스 검토 필수
OrbStack	개인 무료, 상업적 사용 유료	<code>dockerd</code>	macOS 전용	빠르지만 윈도우 사용자와 환경이 갈림
Rancher Desktop	Apache 2.0 완전 무료	<code>Moby(dockerd)</code> / <code>containerd</code> 선택	Win / macOS / Linux	비용 부담 없는 교육 표준 환경

우리가 이 도구를 쓰는 이유는 ① 비용 부담이 전혀 없고 ② 윈도우와 맥 사용자 모두 똑같이 실습할 수 있으며 ③ 도커 내부 엔진 구조를 직접 경험할 수 있기 때문입니다.

설치 및 사전 준비

항목	용도	공식 웹사이트 링크
Rancher Desktop	컨테이너 엔진(Moby/dockerd) 제공	https://rancherdesktop.io/
Docker Hub	공식 Ubuntu 이미지 중앙 저장소	https://hub.docker.com/_/ubuntu
Docker CLI 레퍼런스	명령어·옵션 공식 기술 문서	https://docs.docker.com/reference/cli/docker/
WSL2 안내 (Windows)	윈도우 리눅스 커널 기반 준비	https://learn.microsoft.com/windows/wsl/install

Windows는 `wsl --status`로 활성 여부를 확인합니다. 가상화가 꺼져 있으면 BIOS에서 `Intel VT-x` 또는 `AMD-V`를 켭니다.

Rancher Desktop 설정 체크리스트

순서	설정 위치	권장 설정값	필요한 이유
1	설치 마법사	WSL2 연동 배포판 선택 (Win)	리눅스 커널 구동 기반 확보
2	Preferences > Container Engine	Moby (dockerd)	표준 docker 명령어 실습 전제
3	Preferences > Virtual Machine	메모리 4GB 이상 권장	멀티 컨테이너 실습 대비
4	메인 화면 상태	초기 구동 완료(초록 불) 확인	시동 중에는 명령어 실패

쿠버네티스(Kubernetes)는 본 과정에서 쓰지 않으므로 자원 절약을 위해 비활성화해도 무방합니다.

실습 로드맵

Step	주제	이번 단계에서 직접 확인하는 것
1	Docker 엔진 연결 상태 점검	내 터미널(CLI)과 실제 일꾼 엔진(데몬)의 통신 확인
2	시스템 자원 및 런타임 환경 확인	엔진에 배정된 CPU·메모리와 운영체제 환경 점검
3	Ubuntu 26.04 LTS 이미지 다운로드	원격 저장소에서 최신 공식 리눅스 이미지 내려받기
4	로컬 이미지 목록 및 메타데이터 검증	내 컴퓨터에 저장된 이미지 크기, 태그, CPU 아키텍처 확인
5	대화형 컨테이너 생성 및 쉘 진입	설계도(이미지)로 살아있는 첫 리눅스 컨테이너 띄우기
6	백그라운드 전환 및 재접속 제어	끄지 않고 빠져나오기(Detach)와 다시 들어가기(Attach)

구체적인 명령어 옵션과 실습 세부 가이드는 교안 문서를 함께 보며 단계별로 직접 실행합니다.

명령어 한눈에 보기

엔진 점검 & 이미지 준비 Step 1~4

```
# 1. 엔진 연결 확인
docker version

# 2. 엔진 자원 점검
docker info

# 3. Ubuntu이미지 다운로드
docker pull ubuntu:26.04

# 4. 로컬 이미지 확인
docker images ubuntu
```

컨테이너 실행 & 세션 제어 Step 5~6

```
# 5. 대화형 셸 진입
docker run -it --name linux-practice ubuntu:26.04 bash

# 6. 종료 없이 분리 (Detach)
Ctrl + P, Ctrl + Q

# 분리 확인 후 다시 붙기
docker ps
docker attach linux-practice
```

Step별 이해 포인트

Step	반드시 이해하고 넘어갈 핵심 포인트
1	<code>Client</code> 와 <code>Server</code> 가 둘 다 나와야 성공입니다. <code>Cannot connect ...</code> 에러는 엔진이 아직 구동되지 않았다는 뜻입니다.
2	컨테이너가 쓰는 자원은 내 컴퓨터 전체가 아니라 도커 엔진에 떼어준 몫 안에서만 작동합니다.
3	도커 이미지는 여러 개의 계층 (<code>Layer</code>)으로 나뉘어 내려받습니다. 진행 막대가 여러 줄인 이유입니다.
4	태그(<code>26.04</code>)는 버전 식별자이며, 아키텍처(<code>amd64</code> / <code>arm64</code> , 커널: <code>x86_64</code> / <code>aarch64</code>)는 CPU에 맞게 자동 매핑됩니다.
5	<code>-i</code> (키보드 입력 연결)와 <code>-t</code> (화면 터미널 제공)가 합쳐져야 글자를 주고받는 셸 환경이 열립니다.
6	<code>exit</code> 는 전원 끄기 , <code>Ctrl + P</code> → <code>Ctrl + Q</code> 는 전원 켜두고 문 닫고 나오기 입니다.

핵심 옵션과 세션 제어

옵션 / 단축키	실제 하는 일
<code>-i</code> (interactive)	키보드 입력을 컨테이너 내부로 그대로 전달하도록 표준 입력을 엽니다
<code>-t</code> (tty)	터미널 화면을 만들어 프롬프트(<code>root@<id>:/#</code>)가 정상 표시되게 합니다
<code>--name linux-practice</code>	긴 암호 해시 대신 기억하기 쉬운 고유 이름을 붙여줍니다
<code>Ctrl + P, Ctrl + Q</code>	프로그램을 끄지 않고 백그라운드로 안전하게 빠져나옵니다 (Detach)
<code>docker attach</code>	백그라운드에서 살아서 돌고 있는 컨테이너 화면으로 다시 들어갑니다
<code>docker exec -it ... bash</code>	원래 프로그램과 별개로 컨테이너 안에 새 보조 터미널 창을 하나 더 엽니다

`attach`는 원래 화면에 다시 붙는 것이고, `exec`는 새 창을 띄우는 것입니다. `attach` 상태에서 `exit`를 치면 메인 프로세스가 종료되어 컨테이너가 꺼지므로 반드시 `Ctrl+P, Ctrl+Q`로 빠져나옵니다.

한 장 요약

영역	핵심 요약	비고
컨테이너 본질	호스트 커널 공유 + <code>Namespace</code> / <code>cgroups</code> 프로세스 격리	초 단위 기동, 경량화
아키텍처	CLI(Client) ↔ REST API ↔ Moby Daemon(Server)	양단 연결 점검 (<code>docker version</code>)
3대 구성 요소	이미지(읽기 전용 설계도) → 컨테이너(실행 인스턴스)	불변 이미지 기반 환경 재현
수명주기 제어	<code>exit</code> (프로세스 종료) vs <code>Detach</code> (살려두고 분리)	<code>Ctrl+P, Ctrl+Q</code> 백그라운드 유지

컨테이너는 격리된 프로세스이며, 이미지를 받아 백그라운드로 제어하는 능력이 인프라 실습의 기초입니다.

Appendix

도커 오픈소스의 발전 역사

시기	주요 사건	의의 및 생태계 변화
2013	Docker 오픈소스 공개	dotCloud가 PaaS 기술을 공개하며 컨테이너 열풍 시작
2015	OCI 발족 및 표준화	기업 독점 방지 및 표준 정립, 코어 런타임 <code>runc</code> 기증
2017	Moby 전환 및 containerd 분리	오픈소스 코드를 <code>Moby</code> 로 재편, <code>containerd</code> 를 CNCF에 기증
2021	Docker Desktop 유료 정책	대기업 라이선스 유료화로 Rancher Desktop 등 오픈소스 대안 성장

도커는 단일 프로그램에서 출발해, 현재는 **OCI 표준 런타임(runc)**과 **코어 데몬(containerd)** 위에 상위 도구들이 레고처럼 쌓인 개방형 아키텍처로 진화했습니다.

환경별 배경과 선택 참고

- Windows에서는 리눅스 커널 기능을 제공하는 **WSL2**가 백엔드로 동작합니다. 설치 문제 시 작업 관리자 성능 탭에서 가상화 활성화 여부와 `wsl --status`를 점검합니다.
- CPU 아키텍처는 `x86_64`(인텔/AMD)와 `ARM64`(애플 실리콘)로 나뉩니다. 사양은 `uname -m`으로 확인하며, 다중 아키텍처 이미지는 추후 확장 실습에서 다룹니다.
- Ubuntu `26.04`는 `2026년 4월`에 출시된 LTS 버전입니다. 실습은 장기간 보안 패치와 안정성이 보장되는 최신 공식 LTS 태그로 통일합니다.
- 본 과정은 비용 부담이 없고(Apache 2.0) Windows와 macOS에서 동일하게 표준 Docker CLI를 학습할 수 있는 Rancher Desktop(Moby 엔진)을 기준으로 진행합니다.

핵심 기술 면접 질문

이미지와 컨테이너의 차이

이미지와 컨테이너의 차이는 무엇인가요?

컨테이너와 가상 머신의 차이

컨테이너와 가상 머신의 차이는 무엇인가요?

Windows에서 WSL2의 필요성

Windows 환경에서 Docker 및 Rancher Desktop이 구동될 때 WSL2가 필요한 이유는 무엇인가요?

`docker run -it` 옵션의 역할

`docker run -it` 에서 `-i` 와 `-t` 옵션은 각각 어떤 역할을 하나요?

핵심 기술 면접 질문

Ubuntu LTS의 의미와 서버 사용 이유

Ubuntu 버전 표기(예: **26.04 LTS**)에서 숫자와 LTS의 의미는 무엇이며, 왜 서버 환경에서 LTS를 사용하나요?

컨테이너 삭제 시 데이터 보존 여부

컨테이너를 삭제하면 내부 데이터는 어떻게 되나요?

모범 답변

이미지와 컨테이너의 차이

이미지는 애플리케이션 실행에 필요한 코드, 런타임, 라이브러리를 담은 읽기 전용(Read-Only) 설계도입니다. 컨테이너는 이 이미지를 기반으로 호스트 커널 위에 격리된 프로세스로 실제 구동되는 실행 인스턴스입니다.

컨테이너와 가상 머신의 차이

가상 머신은 하이퍼바이저 위에서 독자적인 게스트 OS 커널을 포함하므로 부팅이 느리고 자원을 많이 소모합니다. 반면 컨테이너는 호스트의 리눅스 커널을 공유하고 네임스페이스와 cgroups로 프로세스만 격리하므로 가볍고 초 단위로 빠르게 시작됩니다.

Windows에서 WSL2의 필요성

리눅스 컨테이너는 리눅스 커널 기능을 직접 활용해야 합니다. Windows 자체는 리눅스 커널이 없기 때문에, 마이크로소프트가 경량 하이퍼바이저 위에 실제 리눅스 커널을 구동하도록 만든 WSL2가 컨테이너 엔진의 백엔드로 동작해야 합니다.

모범 답변

`docker run -it` 옵션의 역할

`-i` (interactive)는 사용자의 표준 입력(stdin)을 유지하여 컨테이너로 키보드 입력을 전달할 수 있게 하고, `-t` (tty)는 가상 터미널 화면을 할당하여 셸 프롬프트(`root@<id>:/#`)가 정상적으로 출력되도록 합니다. 따라서 대화형 터미널 작업 시 필수 조합입니다.

Ubuntu LTS의 의미와 서버 사용 이유

앞의 두 자리는 출시 연도(`26` 년), 뒤의 두 자리는 출시 월(`04` 월)을 뜻합니다. LTS(Long-Term Support)는 2년마다 출시되어 5년간 보안 패치와 버그 수정을 보장하므로, 잦은 마이그레이션과 취약점을 방지하기 위해 프로덕션 서버에서 필수적으로 선택합니다.

컨테이너 삭제 시 데이터 보존 여부

컨테이너 내부 변경 사항 계층은 기본적으로 상태를 저장하지 않는 휘발성(Stateless) 구조입니다. 컨테이너를 삭제하면 내부에서 새로 생성하거나 변경한 데이터도 함께 영구 소멸하므로, 데이터를 보존하려면 호스트 디렉토리를 연결하는 볼륨(Volume)이나 바인드 마운트를 설정해야 합니다.

깊이 알아보기

도커 공식 아키텍처 Docker Overview

엔진 구조와 데몬 동작 원리

<https://docs.docker.com/get-started/docker-overview/>

컨테이너 국제 표준 OCI Runtime Spec

컨테이너 런타임 표준 사양 원문

<https://github.com/opencontainers/runtime-spec>

오픈소스 코어 Moby Project

도커 엔진 오픈소스 업스트림 프로젝트

<https://mobyproject.org/>

리눅스 프로세스 격리 Namespaces

리눅스 커널 프로세스 격리 공식 사양

<https://man7.org/linux/man-pages/man7/namespaces.7.html>

깊이 알아보기

리눅스 자원 제한 Control Groups

리눅스 커널 자원 제어 공식 사양

<https://man7.org/linux/man-pages/man7/cgroups.7.html>

개발 도구 가이드 Rancher Desktop

도커 엔진 설정 및 공식 매뉴얼

<https://docs.rancherdesktop.io/>



- EOF -